

	UNIVERSIDAD NACIONAL DE SAN AGUSTIN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMA	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 1

INFORME DE LABORATORIO

INFORMACIÓN BÁSICA					
ASIGNATURA:	Computación Gráfica, Visión Computacional y Multimedia (E)				
TÍTULO DE LA PRÁCTICA:	<i>Laboratorio 1 - Juego de Básquetbol 2D con Motor de Física</i>				
NÚMERO DE PRÁCTICA:	01	AÑO LECTIVO:	2026	NRO. SEMESTRE:	
FECHA DE PRESENTACIÓN		HORA DE PRESENTACIÓN			
INTEGRANTE (s): - Nina Suyo Diego Claudio				NOTA:	
DOCENTE(s): <i>Mg. Diego Alonso Iquira Becerra</i>					

SOLUCIÓN Y RESULTADOS
I. SOLUCIÓN DE EJERCICIOS/PROBLEMAS Link del video: https://drive.google.com/file/d/1ttk5S6VZSBgY8IKOkg0XCWF8U48Sgjsa/view?usp=sharing Github: https://github.com/DiegoNiS/CGVCM_Lab01-Basquetbol I. OBJETIVOS - Conocer, comprender y analizar aplicaciones multimedia con motor de física. - Implementar un juego 2D en Unity 6 con detección de colisiones y simulación física realista. - Aplicar parámetros físicos modificables en tiempo real para controlar el comportamiento de proyectiles. II. MARCO TEÓRICO Motores de Física Un motor físico (physics engine) es un software capaz de simular sistemas físicos como la dinámica de cuerpo rígido, movimiento de fluidos y elasticidad. En el desarrollo de videojuegos, los motores de

	UNIVERSIDAD NACIONAL DE SAN AGUSTIN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMA	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 2

física permiten que los objetos interactúen de forma realista con el entorno, respondiendo a fuerzas como la gravedad, el impulso y las colisiones.

Unity utiliza el motor de física Box2D para escenas 2D, accesible a través del componente Rigidbody2D. Este componente permite aplicar masa, gravedad, velocidad y fuerzas a los objetos de la escena.

Detección de Colisiones

En Unity 2D, la detección de colisiones se realiza mediante Colliders. Existen dos tipos principales:

- Colliders físicos (BoxCollider2D, CircleCollider2D): generan respuesta física real al contacto entre objetos.
- Triggers (IsTrigger = true): detectan intersección sin generar respuesta física, útiles para zonas de puntuación.

En este laboratorio se utilizaron ambos tipos: colliders físicos en el aro y el suelo, y triggers invisibles (HoopTop y HoopBottom) para detectar si la pelota entra correctamente por el aro.

Lanzamiento de proyectiles

El movimiento parabólico de un proyectil se describe con las siguientes ecuaciones cinemáticas:

$$x(t) = x_0 + v_0 \cos(\theta) t$$

$$y(t) = y_0 + v_0 \sin(\theta) t + \frac{1}{2} g t^2$$

Donde v_0 es la velocidad inicial, θ el ángulo de lanzamiento y g la gravedad. En la implementación, se asigna directamente la velocidad al Rigidbody2D para que la trayectoria predicha coincida exactamente con la trayectoria real del objeto.

III. DESARROLLO DE LA APLICACIÓN

3.1 Descripción General

Se desarrolló un juego de básquetbol 2D en Unity 6 donde el jugador controla un personaje (animal) que lanza una pelota hacia un aro. El juego incluye física real de proyectil, colisiones inteligentes, sistema de puntuación y cambio de personaje tras cada canasta.

3.2 Estructura del Proyecto

- Scripts/Ball.cs — Física y lógica de la pelota
- Scripts/PlayerController.cs — Control del jugador, lanzamiento y trayectoria
- Scripts/GameManager.cs — Puntuación, UI y cambio de personaje

	UNIVERSIDAD NACIONAL DE SAN AGUSTIN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMA	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 3

- Prefabs/Ball — Prefab reutilizable de la pelota
- Sprites/ — Assets visuales (Kenney.nl): animales, pelota, fondo
- Fonts/ — Fuente Press Start 2P para estilo arcade

3.3 Objetos de la Escena

La escena está compuesta por los siguientes GameObjects:

- Player: personaje controlable con Rigidbody2D (gravityScale=0) y el script PlayerController.
- Ball (Prefab): pelota con Rigidbody2D dinámico, CircleCollider2D y script Ball.
- Hoop: contenedor del aro con tablero, RimLeft, RimRight, Pole y los triggers HoopTop/HoopBottom.
- Ground: suelo con BoxCollider2D estático.
- Canvas: UI con textos de puntuación, ángulo, fuerza e instrucciones usando fuente arcade Press Start 2P.
- GameManager: objeto vacío con el script GameManager que coordina el juego.

3.4 Explicación del Código

Ball.cs — Lógica de la Pelota

Este script maneja el estado físico de la pelota y la detección de canasta:

```
public void Launch(Vector2 velocity)
{
    rb.bodyType = RigidbodyType2D.Dynamic;
    rb.gravityScale = 1f;
    rb.linearVelocity = velocity; // Asigna velocidad directamente
    isLaunched = true;
}
```

Se asigna la velocidad directamente (en lugar de AddForce) para que la trayectoria dibujada coincida exactamente con la trayectoria real, independientemente de la masa de la pelota.

La detección de canasta usa dos triggers:

```
void OnTriggerEnter2D(Collider2D other)
{
    // Si entra por HoopTop yendo hacia abajo -> entrada válida
    if (other.CompareTag("HoopTop") && rb.linearVelocity.y < 0)
```

	UNIVERSIDAD NACIONAL DE SAN AGUSTIN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMA	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 4

```

enteredFromAbove = true;

// Si después pasa por HoopBottom -> CANASTA confirmada
if (other.CompareTag("HoopBottom") && enteredFromAbove && !hasScored)
{
    hasScored = true;
    gameManager.AddScore(2);
}
}

```

Este sistema garantiza que solo se otorguen puntos cuando la pelota entra por la parte superior del aro, tal como ocurre en el básquetbol real.

PlayerController.cs — Control del Jugador

Maneja el movimiento horizontal del personaje, los parámetros de lanzamiento y la visualización de la trayectoria predicha:

```

void DrawTrajectory()
{
    float angleRad = launchAngle * Mathf.Deg2Rad;
    float g = Physics2D.gravity.y;
    Vector2 v0 = new Vector2(Mathf.Cos(angleRad), Mathf.Sin(angleRad)) * launchForce;

    for (int i = 0; i < steps; i++) {
        float t = i * 0.04f;
        float x = startPos.x + v0.x * t;
        float y = startPos.y + v0.y * t + 0.5f * g * t * t;
        trajectoryLine.SetPosition(i, new Vector3(x, y, 0));
    }
}

```

La trayectoria usa las mismas ecuaciones cinemáticas que Unity aplica internamente al Rigidbody2D, lo que garantiza que la línea de predicción sea exacta.

Los controles del jugador son:

- A / D o flechas: mover el personaje horizontalmente.
- W / S: aumentar o reducir el ángulo de lanzamiento (10° a 85°).
- Q / E: reducir o aumentar la fuerza de lanzamiento (3 a 25).

	UNIVERSIDAD NACIONAL DE SAN AGUSTIN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMA	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 5

- ESPACIO: lanzar la pelota.
- R: recuperar la pelota manualmente.

GameManager.cs — Gestión del Juego

Coordina la puntuación, la UI y el cambio de personaje tras cada canasta:

```
public void AddScore(int points)
{
    score += points;
    UpdateUI();
    ShowMessage("¡CANASTA! +" + points);
    ChangeCharacter(); // Cambia al siguiente animal
}

void ChangeCharacter()
{
    currentCharacterIndex = (currentCharacterIndex + 1) % characterSprites.Length;
    playerRenderer.sprite = characterSprites[currentCharacterIndex];
}
```

El arreglo characterSprites contiene los 10 animales del pack de Kenney.nl. Después de cada canasta, el índice avanza cíclicamente mostrando un nuevo personaje.

IV. SISTEMA DE COLISIONES

El juego implementa tres tipos de colisiones:

4.1 Colisión Pelota–Suelo y Pelota–Aro

La pelota tiene un CircleCollider2D que interactúa físicamente con el suelo (BoxCollider2D estático) y con los bordes del aro (RimLeft y RimRight). Al rebotar en estos objetos, Unity calcula automáticamente la respuesta física usando las propiedades del Rigidbody2D.

4.2 Detección de Canasta (Triggers)

Se usan dos zonas trigger invisibles superpuestas en el aro:

- HoopTop: zona superior del aro. Si la pelota la atraviesa moviéndose hacia abajo ($velocityY < 0$), se registra entrada válida.

	UNIVERSIDAD NACIONAL DE SAN AGUSTIN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMA	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 6

- HoopBottom: zona inferior del aro. Si la pelota la atraviesa después de haber pasado HoopTop, se confirma la canasta y se suman 2 puntos.

Este sistema de doble trigger evita que se otorguen puntos cuando la pelota golpea el aro desde abajo o desde los lados.

V. RESULTADOS

Se logró desarrollar un juego de básquetbol 2D completamente funcional que cumple con todos los requisitos del laboratorio:

- Motor de física implementado correctamente con Rigidbody2D y gravedad real.
- Parámetros físicos modificables en tiempo real: ángulo (10° – 85°) y fuerza (3–25).
- Trayectoria predicha que coincide exactamente con el vuelo real de la pelota.
- Detección de canasta inteligente: solo puntúa si la pelota entra por la parte superior del aro.
- Sistema de puntuación visible en pantalla con fuente arcade.
- Cambio de personaje tras cada canasta (10 animales disponibles).
- Assets visuales de calidad usando sprites gratuitos de Kenney.nl.

II. SOLUCIÓN DEL CUESTIONARIO

– No hay cuestionario –

III. CONCLUSIONES

El desarrollo de este laboratorio permitió comprender en la práctica cómo funcionan los motores de física en aplicaciones multimedia. La implementación del sistema de colisiones con triggers dobles demostró que la detección precisa de eventos requiere diseñar zonas de detección específicas más allá de los colliders físicos básicos.

La coincidencia entre la trayectoria predicha y la real se logró al asignar la velocidad directamente al Rigidbody2D en lugar de aplicar fuerzas con impulso, lo que evidencia la importancia de entender cómo Unity aplica internamente las leyes cinemáticas.

El uso de assets gratuitos de Kenney.nl y la fuente Press Start 2P permitió lograr una estética arcade coherente sin invertir tiempo en la creación de arte propio.

	UNIVERSIDAD NACIONAL DE SAN AGUSTIN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMA	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 7

REFERENCIAS Y BIBLIOGRAFÍA

- [1] Gordon, V. S., & Clevenger, J. L. (2020). Computer Graphics Programming in OpenGL with C++. Stylus Publishing, LLC.
- [2] D. P. Kothari, G. Awari, D. Shrimankar, A. Bhende (2019). Mathematics for Computer Graphics and Game Programming: A Self-Teaching Introduction.
- [3] Unity Technologies. (2024). Unity 6 Documentation — Rigidbody2D.
<https://docs.unity3d.com/6000.0/Documentation/ScriptReference/Rigidbody2D.html>
- [4] Kenney.nl. (2024). Free Game Assets. <https://kenney.nl/assets>